

**JSPM'S, BHIVARABAI SAWANT INSTITUTE OF
TECHNOLOGY
& RESEARCH
DEPARTMENT OF INFORMATION TECHNOLOGY**

**A PRELIMINARY PROJECT REPORT
ON
SKILLCHAIN: DECENTRALIZED
FREELANCE MARKETPLACE**

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN PARTIAL FULFILLMENT OF REQUIREMENTS
FOR THE AWARD OF THE DEGREE

BACHELOR OF ENGINEERING (INFORMATION TECHNOLOGY)

SUBMITTED BY

Avadhoot Smart	Exam No:
Aditya Kumbhar	Exam No:
Atharva Rokade	Exam No:
Pranav Pagar	Exam No:

Under the guidance of
Dr. Vinod Wadne

2025-2026

CERTIFICATE

This is to certify that the project report entitled “**SkillChain: Decentralized Freelance Marketplace**” submitted by

Avadhoot Smart

Exam No:

Aditya Kumbhar

Exam No:

Atharva Rokade

Exam No:

Pranav Pagar

Exam No:

are bonafide students of this institute and the work has been carried out by them under the supervision of Prof. Vinod Wadne and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of Bachelor of Engineering (Information Technology).

Dr. Vinod Wadne
Project Guide

Prof. Rekha Kotwal
Project Coordinator

Dr. Vinod Wadne
HOD

Dr. T. K. Nagaraj
Principal

External Examiner

Place: Wagholi, Pune

Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who contributed to the successful completion of this project.

First and foremost, we thank our project guide, Dr. Vinod Wadne, for their invaluable guidance, constant support, and constructive feedback throughout the development of SkillChain. Their expertise in blockchain technology and distributed systems helped us navigate complex technical challenges.

We extend our heartfelt thanks to Prof. Rekha Kotwal, our Project Coordinator, for coordinating the project activities and ensuring timely completion of milestones.

We are grateful to Dr. Vinod Wadne, Head of the Department of Information Technology, for providing us with the necessary resources and infrastructure to carry out this project.

We also thank Dr. T. K. Nagaraj, Principal of JSPM's BSIOTR, for his encouragement and for fostering an environment conducive to innovation and research.

Our sincere appreciation goes to all faculty members of the Department of Information Technology for their continuous support and valuable suggestions.

Finally, we thank our families and friends for their unwavering support and motivation throughout this journey.

Avadhoot Smart
Aditya Kumbhar
Atharva Rokade
Pranav Pagar

ABSTRACT

SkillChain is a decentralized freelance marketplace built on Ethereum blockchain technology that eliminates intermediaries and establishes trust through smart contracts. The platform connects clients and freelancers directly, utilizing IPFS for decentralized data storage and implementing an escrow-based payment system.

The traditional freelance industry faces significant challenges including high commission fees (typically 10-20%), payment delays of 5-14 days, lack of transparency in dispute resolution, and data privacy concerns due to centralized storage. These issues affect millions of freelancers and clients worldwide, reducing earnings and creating friction in the gig economy.

This project presents the complete architecture, design decisions, algorithms, and implementation details of the SkillChain platform. The system addresses key challenges in traditional freelance platforms through blockchain-based solutions including direct peer-to-peer transactions, smart contract escrow for secure payment holding and conditional release, decentralized IPFS storage for data permanence and censorship resistance, transparent operations with all transactions verifiable on the blockchain, and minimal blockchain gas costs compared to traditional platform fees.

The platform implements three interconnected smart contracts: UserProfile Contract for managing user identities and roles, JobsContract for handling job creation, escrow management, and payment release, and ProposalsContract for managing proposal submission and approval workflow. The frontend is built using modern web technologies including Next.js, React, TypeScript, and Tailwind CSS, with Web3 integration through wagmi and RainbowKit.

Security considerations include re-entrancy protection through the Checks-Effects-Interactions pattern, comprehensive input validation, and multi-party approval for fund release. The system has been thoroughly tested using Foundry framework with 100% coverage of contract functions.

The project demonstrates the feasibility of decentralized freelance marketplaces and provides a foundation for future enhancements including Layer 2 integration, reputation systems, and decentralized dispute resolution.

Keywords: Blockchain, Smart Contracts, Decentralized Application, Ethereum, IPFS,

Contents

Certificate	i
Acknowledgement	ii
Abstract	iii
List of Abbreviations	viii
List of Figures	ix
List of Tables	1
1 Introduction	2
1.1 Background	2
1.1.1 Challenges in Traditional Freelance Platforms	2
1.2 Project Overview	2
1.2.1 Key Features	3
1.3 Problem Definition	3
1.3.1 Challenges in Traditional Freelance Platforms	3
1.3.2 Technical Challenges Addressed	3
1.4 Objectives	4
2 Literature Survey	5
2.1 Blockchain in Freelance Markets	5
2.2 IPFS for Decentralized Storage	5
2.3 Escrow Mechanisms	6
2.4 Related Work	6
2.4.1 Existing Blockchain Freelance Platforms	6
2.4.2 Limitations of Existing Solutions	6
3 Software Requirements Specification	8
3.1 Project Scope	8
3.1.1 In Scope	8
3.1.2 Out of Scope	8

3.2	User Classes and Characteristics	9
3.2.1	Clients	9
3.2.2	Freelancers	9
3.3	Assumptions and Dependencies	9
3.3.1	Assumptions	9
3.3.2	Dependencies	9
3.4	Functional Requirements	10
3.4.1	User Profile Management	10
3.4.2	Job Management	10
3.4.3	Proposal Management	10
3.4.4	Payment and Escrow	10
3.5	External Interface Requirements	10
3.5.1	User Interfaces	10
3.5.2	Hardware Interfaces	10
3.5.3	Software Interfaces	11
3.5.4	Communication Interfaces	11
3.6	Non-Functional Requirements	11
3.6.1	Performance Requirements	11
3.6.2	Safety Requirements	11
3.6.3	Security Requirements	11
3.6.4	Software Quality Attributes	12
3.7	System Requirements	12
3.7.1	Database Requirements	12
3.7.2	Software Requirements	13
3.7.3	Hardware Requirements	13
3.8	Analysis Models	13
3.8.1	Data Flow	13
3.8.2	System Implementation Plan	14
4	System Design	15
4.1	System Architecture	15
4.1.1	High-Level Architecture	15
4.1.2	Layer 1: Presentation Layer	15
4.1.3	Layer 2: Application Layer	16
4.1.4	Layer 3: Blockchain Layer	16
4.1.5	Layer 4: Storage Layer	16
4.2	Data Flow Diagrams	17
4.2.1	Level 0: Context Diagram	17
4.2.2	Level 1: FirstLevel Decomposition	18

4.2.3	Level 2: Detailed Process Diagrams	19
4.2.4	Job Creation Flow	23
4.2.5	Proposal and Hiring Flow	23
4.2.6	Completion and Payment Flow	23
4.3	UML Diagrams	24
4.3.1	Use Case Diagram	24
4.3.2	Class Diagram	25
4.3.3	Sequence Diagram: Job Creation	26
5	Implementation Details	27
5.1	Smart Contract Implementation	27
5.1.1	UserProfile Contract	27
5.1.2	JobsContract	28
5.1.3	ProposalsContract	28
5.2	Frontend Implementation	29
5.2.1	Project Structure	29
5.2.2	State Management	29
5.2.3	IPFS Integration	29
5.2.4	Blockchain Configuration	30
5.2.5	Gas Optimization and Access Control	30
6	Testing and Validation	31
6.1	Test Suite	31
6.1.1	Test Coverage Summary	31
6.2	Test Execution	31
6.2.1	Test Coverage Results	31
6.3	Security Audit Checklist	32
6.4	Integration Testing	32
6.5	Performance Testing	33
6.5.1	Gas Cost Analysis	33
6.5.2	Load Testing	33
6.6	Validation Results	33
7	Other Specifications	34
7.1	Advantages	34
7.2	Limitations	34
7.3	Applications	35
8	Conclusion	36
8.1	Summary	36

8.2	Technical Achievements	36
8.3	Learning Outcomes	37
8.4	Impact and Concluding Remarks	37
9	Future Work	38
9.1	Overview	38
9.2	Advanced Features	38
9.3	Scalability Improvements	39
9.4	Business Enhancements	39
9.5	Roadmap Summary	40
A	Deployment and Configuration	41
A.1	Deployment Configuration	41
A.1.1	Deploy Script Summary	41
A.1.2	Environment Configuration	41
A.1.3	Deployment Workflow	41
A.2	Pinata IPFS Setup	42
A.3	Operational Considerations	42
	Bibliography	42

LIST OF ABBREVIATIONS

Abbreviation	Full Form
DApp	Decentralized Application
IPFS	InterPlanetary File System
CID	Content Identifier
ETH	Ether (Ethereum cryptocurrency)
EVM	Ethereum Virtual Machine
RPC	Remote Procedure Call
JWT	JSON Web Token
UI	User Interface
UX	User Experience
REST	Representational State Transfer
API	Application Programming Interface
SQL	Structured Query Language
CSS	Cascading Style Sheets
DOM	Document Object Model
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
JSON	JavaScript Object Notation
SDK	Software Development Kit
IDE	Integrated Development Environment
CI/CD	Continuous Integration/Continuous Deployment
MVVM	Model-View-ViewModel
SSR	Server-Side Rendering
SEO	Search Engine Optimization
PWA	Progressive Web Application
L2	Layer 2 (scaling solution)
PoS	Proof of Stake
PoW	Proof of Work

LIST OF FIGURES

List of Figures

4.1	Level 0 Data Flow Diagram - Context Diagram	17
4.2	Level 1 Data Flow Diagram - Process Decomposition	18
4.3	Level 2 DFD - Job Creation Process	19
4.4	Level 2 DFD - Proposal Management Process	22
4.5	Use Case Diagram	24
4.6	Sequence Diagram - Job Creation Flow	26

LIST OF TABLES

List of Tables

1.1	Comparison of Traditional vs. SkillChain Solutions	3
3.1	Software Requirements	13
4.1	System Architecture Layers	15
4.2	UserProfile Contract Structure	25
4.3	JobsContract Structure	25
4.4	ProposalsContract Structure	25
5.1	UserProfile Functions	27
5.2	JobsContract Functions	28
6.1	Test Cases by Contract	31
6.2	Test Coverage Summary	32
6.3	Security Audit Results	32
6.4	Gas Cost Summary	33
6.5	Requirements Validation	33

1 INTRODUCTION

1.1 BACKGROUND

The gig economy has experienced exponential growth over the past decade, with freelance platforms connecting millions of clients and freelancers worldwide. Traditional centralized platforms like Upwork, Fiverr, and Freelancer.com have become essential infrastructure for the modern workforce. However, these platforms impose significant challenges that affect both clients and freelancers.

1.1.1 Challenges in Traditional Freelance Platforms

- **High Service Fees:** Traditional platforms charge 10-20% commission on every transaction, significantly reducing freelancer earnings and increasing costs for clients.
- **Payment Delays:** Withdrawal periods typically range from 5-14 days, creating cash flow problems for freelancers who need immediate access to their earnings.
- **Lack of Transparency:** Fee structures and dispute resolution processes are often opaque, leaving users uncertain about platform decisions.
- **Data Privacy Concerns:** Centralized storage of sensitive user data creates vulnerability to breaches and unauthorized access.
- **Censorship Risk:** Platform operators can suspend accounts arbitrarily, potentially locking users out of their earnings and reputation.

Blockchain technology offers a paradigm shift by enabling peer-to-peer interactions without trusted intermediaries. Smart contracts can automate escrow, payment release, and dispute resolution in a transparent, immutable manner.

1.2 PROJECT OVERVIEW

SkillChain is a decentralized application (DApp) that revolutionizes the freelance marketplace through innovative blockchain-based solutions.

1.2.1 Key Features

- **Direct Peer-to-Peer Transactions:** Eliminating intermediaries reduces costs and speeds up payments.
- **Smart Contract Escrow:** Secure payment holding with conditional release based on mutual approval.
- **Decentralized Storage:** IPFS ensures data permanence and censorship resistance.
- **Transparent Operations:** All transactions are verifiable on the public blockchain.
- **Reduced Fees:** Minimal blockchain gas costs compared to platform fees (typically less than 1%).

1.3 PROBLEM DEFINITION

1.3.1 Challenges in Traditional Freelance Platforms

Table 1.1: Comparison of Traditional vs. SkillChain Solutions

Challenge	Impact	SkillChain Solution
High Commission Fees	10-20% of earnings lost	Direct blockchain transactions
Payment Delays	5-14 day withdrawal periods	Immediate smart contract release
Lack of Transparency	Unclear fee structures	Open-source smart contracts
Data Privacy	Centralized data storage	IPFS distributed storage
Dispute Resolution	Arbitrary decisions	Mutual consensus mechanism
Censorship Risk	Account suspension	Decentralized identity

1.3.2 Technical Challenges Addressed

1. **Scalability:** Using IPFS for off-chain data storage reduces blockchain bloat and transaction costs.
2. **Cost Efficiency:** Optimized gas usage through efficient data structures and minimal on-chain storage.
3. **User Experience:** Abstracting blockchain complexity through intuitive UI and wallet integrations.

-
4. **Security:** Multi-party approval for fund release prevents fraud and ensures fair transactions.

1.4 OBJECTIVES

The primary objectives of the SkillChain project are:

1. Develop a fully functional decentralized freelance marketplace with seamless user experience.
2. Implement secure escrow mechanism with multi-signature approval for payment release.
3. Create intuitive user interfaces for both clients and freelancers with role-based access.
4. Ensure data privacy through off-chain IPFS storage while maintaining data integrity.
5. Establish fair dispute resolution through mutual consensus mechanism.
6. Achieve significant cost reduction compared to traditional platforms.
7. Demonstrate the feasibility of blockchain-based peer-to-peer marketplaces.

2 LITERATURE SURVEY

2.1 BLOCKCHAIN IN FREELANCE MARKETS

Blockchain technology enables trustless interactions through immutable ledgers and programmable smart contracts. The decentralized nature of blockchain eliminates the need for trusted third parties, allowing direct peer-to-peer transactions with cryptographic security guarantees.

Ethereum's Turing-complete platform allows implementation of complex business logic through smart contracts. These self-executing contracts automatically enforce agreement terms without intermediaries, reducing costs and eliminating single points of failure.

Key advantages of blockchain for freelance platforms include:

- **Immutability:** Once recorded, transactions cannot be altered or deleted.
- **Transparency:** All transactions are publicly verifiable on the blockchain.
- **Decentralization:** No single entity controls the platform or user funds.
- **Programmability:** Smart contracts enable complex conditional logic.

2.2 IPFS FOR DECENTRALIZED STORAGE

The InterPlanetary File System (IPFS) provides content-addressed storage, ensuring data integrity and availability. Unlike traditional location-based addressing (HTTP URLs), IPFS uses content-based addressing where each piece of content is identified by its cryptographic hash.

IPFS CIDs (Content Identifiers) serve as permanent references to stored content. This approach offers several benefits:

- **Data Integrity:** Content cannot be tampered with without changing the CID.
- **Censorship Resistance:** Content is distributed across multiple nodes.
- **Deduplication:** Identical content is stored only once.

-
- **Offline Availability:** Content can be accessed without internet connectivity once cached.

2.3 ESCROW MECHANISMS

Multi-signature escrow systems require multiple parties to approve transactions, reducing counterparty risk. In the context of freelance platforms, escrow ensures that:

1. Client funds are locked before work begins.
2. Freelancer is guaranteed payment upon successful completion.
3. Both parties must agree before funds are released.
4. No single party can unilaterally access escrowed funds.

Smart contract-based escrow automates execution based on predefined conditions, eliminating manual intervention while maintaining security. The mutual approval mechanism ensures fairness and prevents disputes.

2.4 RELATED WORK

Several platforms have attempted to address freelance marketplace challenges through blockchain technology:

2.4.1 Existing Blockchain Freelance Platforms

- **Ethlance:** One of the first decentralized freelance platforms on Ethereum. While innovative, it suffers from high gas costs and limited functionality.
- **Braintrust:** A decentralized talent network with lower fees but still relies on centralized components.
- **Gitcoin:** Focused on open-source development bounties with a unique quadratic funding model.

2.4.2 Limitations of Existing Solutions

Current solutions face several challenges:

- High transaction costs on Ethereum mainnet.
- Complex user onboarding requiring technical knowledge.
- Limited dispute resolution mechanisms.

-
- Poor user interface compared to traditional platforms.
 - Lack of comprehensive features (profiles, portfolios, messaging).

SkillChain addresses these limitations through optimized smart contracts, intuitive UI/UX, and comprehensive feature set while maintaining full decentralization.

3 SOFTWARE REQUIREMENTS SPECIFICATION

3.1 PROJECT SCOPE

3.1.1 In Scope

The SkillChain platform encompasses the following features and capabilities:

- User registration and profile management for both clients and freelancers.
- Job posting with detailed descriptions, budgets, and requirements.
- Proposal submission and management system.
- Smart contract-based escrow for secure payment holding.
- Mutual approval mechanism for job completion and payment release.
- IPFS integration for decentralized data storage.
- Web3 wallet integration for blockchain interactions.
- Real-time job and proposal tracking dashboard.
- Role-based access control (Client vs. Freelancer).

3.1.2 Out of Scope

The following features are not included in the current implementation but are planned for future releases:

- Decentralized dispute resolution with third-party arbitration.
- On-chain reputation and rating system.
- Multi-token support (ERC-20 tokens).
- Layer 2 scaling solutions (Arbitrum, Optimism).
- Push notifications and real-time messaging.
- Milestone-based payment schedules.
- Time tracking integration.

3.2 USER CLASSES AND CHARACTERISTICS

3.2.1 Clients

Users who post jobs and hire freelancers. They require a basic understanding of cryptocurrency wallets, use the platform occasionally to frequently for job posting, and operate at a high security level because they handle ETH for escrow deposits.

3.2.2 Freelancers

Users who submit proposals and complete work. They require a basic understanding of cryptocurrency wallets, use the platform regularly for proposal submissions and work delivery, and operate at a medium security level (they receive payments but do not deposit).

3.3 ASSUMPTIONS AND DEPENDENCIES

3.3.1 Assumptions

1. Users have access to modern web browsers with Web3 capabilities.
2. Users possess or can create cryptocurrency wallets (MetaMask, Rainbow, etc.).
3. Users have basic understanding of blockchain transactions and gas fees.
4. Ethereum network is available and operational.
5. IPFS network is accessible for content retrieval.
6. Users have stable internet connectivity.

3.3.2 Dependencies

1. Ethereum blockchain network (local, testnet, or mainnet).
2. IPFS/Pinata service for decentralized storage.
3. Wallet providers (MetaMask, Rainbow, etc.).
4. Web3 libraries (wagmi, viem, RainbowKit).
5. Node.js runtime environment.
6. Modern web browsers supporting ES6+ JavaScript.

3.4 FUNCTIONAL REQUIREMENTS

3.4.1 User Profile Management

The system shall allow users to create profiles with unique usernames and role selection (Client or Freelancer). Profile data is stored on IPFS and referenced via CID, with uniqueness enforcement and update capabilities (FR-1.1 to FR-1.5).

3.4.2 Job Management

Clients can create jobs with title, description, and budget, requiring ETH deposit equal to the budget amount. Funds are locked in escrow, with job status tracking through Open, In Progress, and Completed states (FR-2.1 to FR-2.6).

3.4.3 Proposal Management

Freelancers submit proposals for open jobs with details stored on IPFS. Clients can view and approve proposals, which automatically hires the freelancer. The system prevents duplicate proposals from the same freelancer for one job (FR-3.1 to FR-3.6).

3.4.4 Payment and Escrow

The escrow system holds client funds upon job creation. Both freelancer and client must mark the job complete for fund release, which transfers ETH directly to the freelancer's wallet. The system prevents premature fund release (FR-4.1 to FR-4.6).

3.5 EXTERNAL INTERFACE REQUIREMENTS

3.5.1 User Interfaces

- **UI-1:** Web-based interface accessible via modern browsers.
- **UI-2:** Responsive design supporting desktop and mobile devices.
- **UI-3:** Intuitive navigation with clear role-based menus.
- **UI-4:** Real-time feedback for blockchain transactions.
- **UI-5:** Wallet connection interface with multiple provider support.

3.5.2 Hardware Interfaces

- **HI-1:** Standard computing devices (PC, laptop, tablet, smartphone).
- **HI-2:** Internet connectivity (minimum 1 Mbps).
- **HI-3:** No specialized hardware required.

3.5.3 Software Interfaces

- **SI-1:** Web3 wallet browser extensions (MetaMask, Rainbow, etc.).
- **SI-2:** Ethereum JSON-RPC endpoints (Infura, Alchemy, local node).
- **SI-3:** IPFS gateway for content retrieval.
- **SI-4:** Pinata API for IPFS content pinning.

3.5.4 Communication Interfaces

- **CI-1:** HTTPS for web application communication.
- **CI-2:** WebSocket for real-time updates (future enhancement).
- **CI-3:** Ethereum JSON-RPC for blockchain interactions.
- **CI-4:** IPFS HTTP API for storage operations.

3.6 NON-FUNCTIONAL REQUIREMENTS

3.6.1 Performance Requirements

- **NFR-1.1:** Page load time shall not exceed 3 seconds on standard broadband.
- **NFR-1.2:** Smart contract interactions shall complete within 30 seconds (dependent on network).
- **NFR-1.3:** IPFS content retrieval shall complete within 5 seconds.
- **NFR-1.4:** System shall support up to 1000 concurrent users.

3.6.2 Safety Requirements

- **NFR-2.1:** Smart contracts shall implement re-entrancy guards.
- **NFR-2.2:** User funds shall be protected against unauthorized access.
- **NFR-2.3:** System shall validate all inputs to prevent attacks.
- **NFR-2.4:** Failed transactions shall not result in fund loss.

3.6.3 Security Requirements

- **NFR-3.1:** All transactions require cryptographic signatures.
- **NFR-3.2:** Smart contracts shall follow established security patterns.

-
- **NFR-3.3:** User data shall be encrypted during IPFS storage.
 - **NFR-3.4:** Wallet connections shall use secure protocols.
 - **NFR-3.5:** Private keys shall never be stored or transmitted.

3.6.4 Software Quality Attributes

- **Maintainability:** Modular architecture with clear separation of concerns.
- **Scalability:** Horizontal scaling capability for frontend components.
- **Usability:** Intuitive UI requiring minimal blockchain knowledge.
- **Reliability:** 99.9% uptime for web application.
- **Portability:** Cross-platform compatibility (Windows, macOS, Linux, mobile).

3.7 SYSTEM REQUIREMENTS

3.7.1 Database Requirements

The system uses IPFS for decentralized storage with the following data structures:

- Profile metadata stored as JSON objects.
- Job details stored as JSON objects with IPFS CIDs.
- Proposal details stored as JSON objects.
- Deliverables stored as files or JSON objects.
- Blockchain stores only references (CIDs) and critical state.

3.7.2 Software Requirements

Table 3.1: Software Requirements

Component	Technology	Version
Frontend Framework	Next.js	16.1.1
UI Library	React	19.2.3
Styling	Tailwind CSS	4.1.18
State Management	Zustand	5.0.9
Data Fetching	TanStack Query	5.90.16
Blockchain	wagmi	3.4.1
Wallet	RainbowKit	2.2.10
Smart Contracts	Solidity	0.8.24+
Development	Foundry	Latest
Storage	IPFS/Pinata	-

3.7.3 Hardware Requirements

- **Development:**

- CPU: Intel Core i5 or equivalent
- RAM: 8 GB minimum, 16 GB recommended
- Storage: 50 GB free space
- Internet: Broadband connection

- **End Users:**

- Device: Computer, tablet, or smartphone
- Browser: Chrome, Firefox, Safari, Edge (latest versions)
- Internet: Stable connection (1 Mbps minimum)

3.8 ANALYSIS MODELS

3.8.1 Data Flow

The system follows a multi-layer data flow:

1. User interacts with frontend interface.
2. Frontend communicates with smart contracts via Web3.
3. Smart contracts interact with blockchain state.

-
4. Frontend reads/writes metadata to IPFS.
 5. Blockchain events trigger UI updates.

3.8.2 System Implementation Plan

1. **Phase 1:** Smart contract development and testing.
2. **Phase 2:** Frontend development and Web3 integration.
3. **Phase 3:** IPFS integration and content management.
4. **Phase 4:** Testing and security audit.
5. **Phase 5:** Deployment and documentation.

4 SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The SkillChain platform follows a multi-layer architecture that separates concerns and enables scalability, maintainability, and security.

4.1.1 High-Level Architecture

Table 4.1: System Architecture Layers

Layer	Components	Responsibilities
Presentation Layer	Next.js Frontend, Client UI, Freelancer UI, Dashboard	User interface rendering and interaction
Application Layer	Wagmi hooks, Zustand state, React Query, IPFS integration	Business logic and data management
Blockchain Layer	Ethereum Network, Smart Contracts	Decentralized computation and storage
Storage Layer	IPFS/Pinata	Decentralized content storage

4.1.2 Layer 1: Presentation Layer

The presentation layer is built using Next.js 16 with the App Router architecture. It provides:

- Responsive user interfaces for clients and freelancers.
- Role-based dashboard views.
- Wallet connection management via RainbowKit.
- Real-time transaction status updates.
- Form validation and error handling.

4.1.3 Layer 2: Application Layer

The application layer handles business logic and state management:

- **Wagmi Hooks:** Ethereum blockchain interactions (read/write operations).
- **Zustand:** Global state management for user sessions and application state.
- **React Query:** Server state management with caching and synchronization.
- **IPFS Integration:** Content upload and retrieval via Pinata API.

4.1.4 Layer 3: Blockchain Layer

The blockchain layer consists of three interconnected smart contracts:

1. **UserProfile Contract:** Manages user registration and profiles.
2. **JobsContract:** Handles job creation, escrow, and payment release.
3. **ProposalsContract:** Manages proposal submission and approval.

4.1.5 Layer 4: Storage Layer

IPFS provides decentralized storage for:

- User profile metadata.
- Job descriptions and requirements.
- Proposal details and pricing.
- Work deliverables and attachments.

4.2 DATA FLOW DIAGRAMS

4.2.1 Level 0: Context Diagram

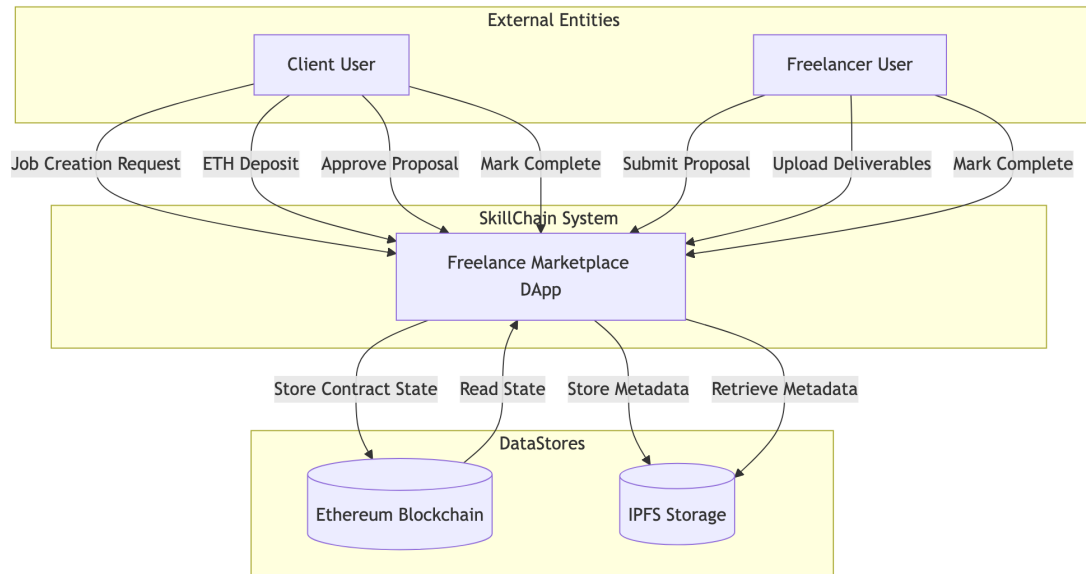


Figure 4.1: Level 0 Data Flow Diagram - Context Diagram

The Level 0 DFD shows the SkillChain system as a single process interacting with external entities (Clients, Freelancers) and data stores (Ethereum Blockchain, IPFS Storage). The system receives requests from users, processes them through smart contracts, and stores/retrieves data from the blockchain and IPFS.

4.2.2 Level 1: FirstLevel Decomposition

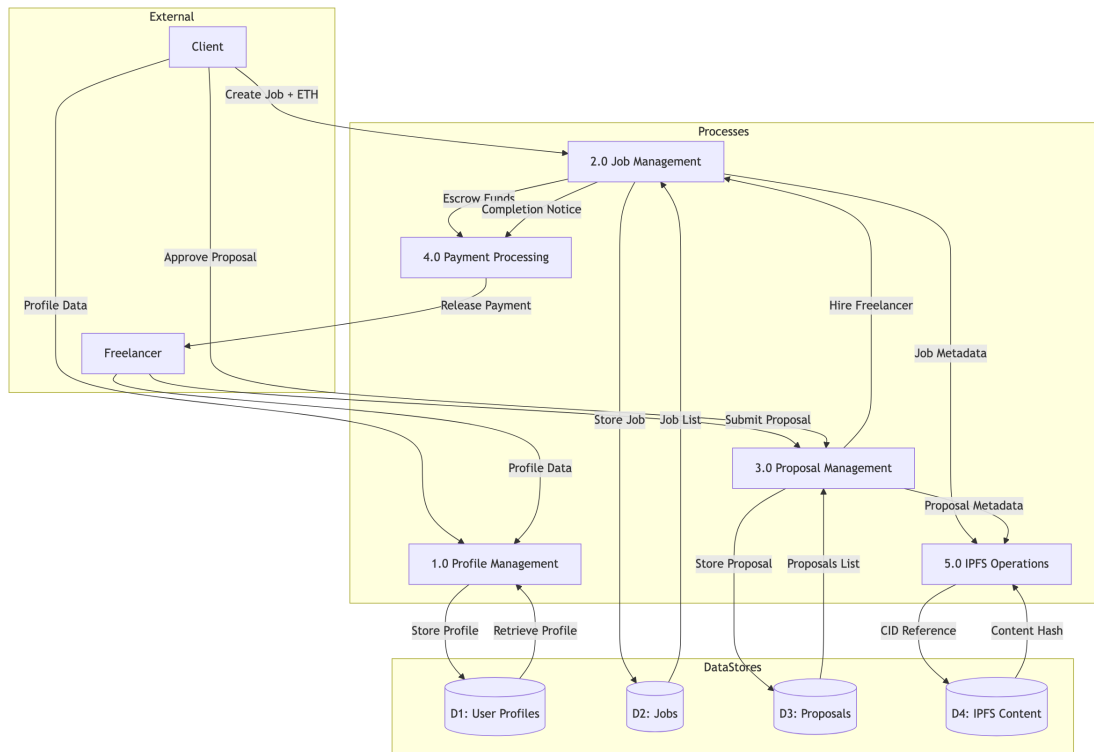


Figure 4.2: Level 1 Data Flow Diagram - Process Decomposition

The Level1 DFD decomposes the system into five major processes: Profile Management (1.0), Job Management (2.0), Proposal Management (3.0), Payment Processing (4.0), and IPFS Operations (5.0). Each process interacts with specific data stores and external actors.

4.2.3 Level 2: Detailed Process Diagrams

Job Creation Process

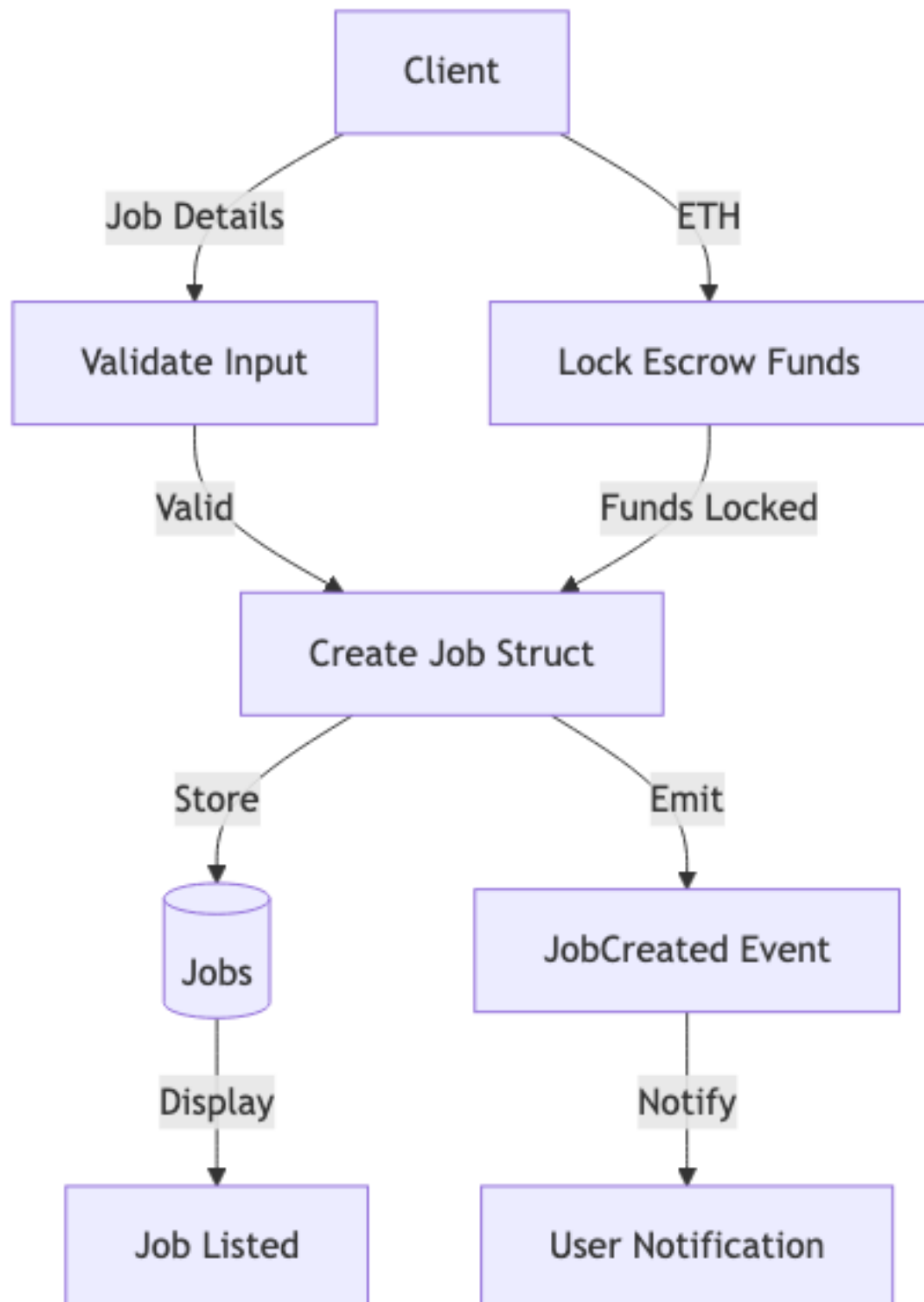


Figure 4.3: Level 2 DFD - Job Creation Process

Process 2.0 Job Management is further decomposed into subprocesses for validation, ID generation, escrow locking, storage, and event emission.

Proposal Management Process

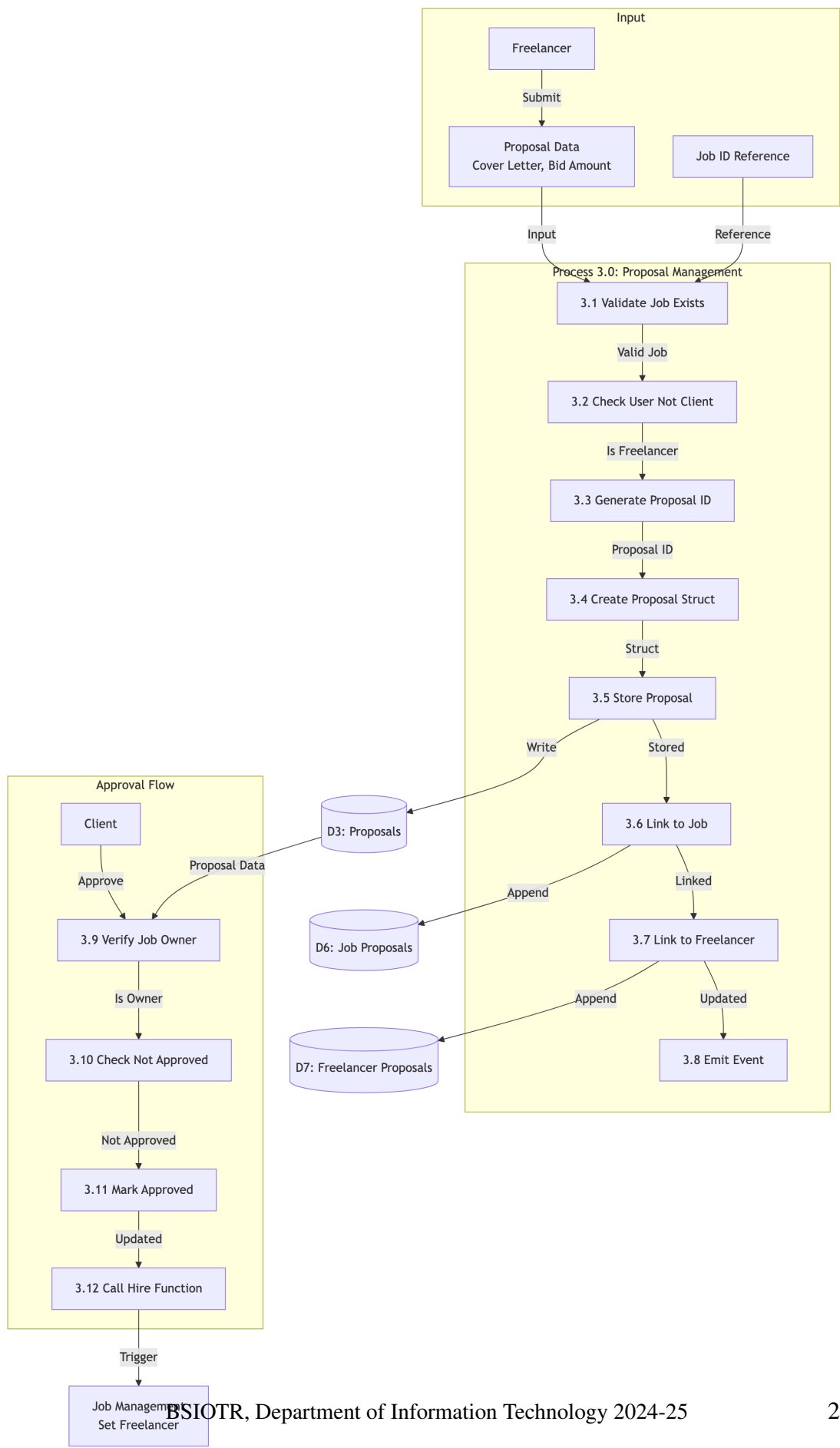


Figure 4.4: Level 2 DFD - Proposal Management Process

Process 3.0 Proposal Management handles submission validation, storage, job linking, and the approval workflow that triggers freelancer hiring.

4.2.4 Job Creation Flow

1. Client fills job creation form with title, description, budget.
2. Frontend uploads metadata to IPFS and receives CID.
3. Client confirms transaction with ETH deposit.
4. Smart contract creates job with IPFS CID and locks funds.
5. Job is indexed and displayed in job listings.

4.2.5 Proposal and Hiring Flow

1. Freelancer views available jobs.
2. Freelancer submits proposal with cover letter and bid.
3. Proposal metadata uploaded to IPFS.
4. Smart contract stores proposal reference.
5. Client reviews proposals and approves selected freelancer.
6. Approval automatically triggers freelancer hiring.
7. Job status changes to “In Progress”.

4.2.6 Completion and Payment Flow

1. Freelancer completes work and uploads deliverables to IPFS.
2. Freelancer marks job complete via smart contract.
3. Client reviews deliverables.
4. Client marks job complete via smart contract.
5. Upon mutual approval, funds are released to freelancer.
6. Job marked as completed.

4.3 UML DIAGRAMS

4.3.1 Use Case Diagram

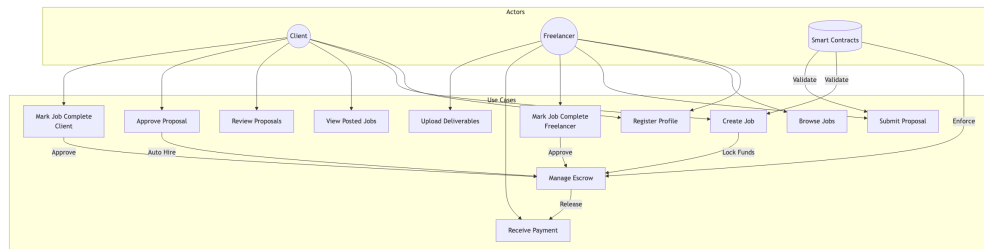


Figure 4.5: Use Case Diagram

Actors:

- Client
- Freelancer
- System (Smart Contracts)

Client Use Cases:

- Register/Create Profile
- Create Job
- View Posted Jobs
- Review Proposals
- Approve Proposal/Hire Freelancer
- Mark Job as Completed

Freelancer Use Cases:

- Register/Create Profile
- Browse Jobs
- Submit Proposal
- View Submitted Proposals
- Submit Deliverables
- Mark Job as Completed

4.3.2 Class Diagram

The smart contract layer can be modelled as three loosely coupled classes, each with a single domain-level struct and a small public API. The tables below summarise the attributes and operations of each contract; together they form the conceptual class diagram of the system.

Table 4.2: UserProfile Contract Structure

Element	Description
Struct Profile	userID, username, cid, role, userAddress
setProfile	Creates or updates the caller's profile, enforcing user-name uniqueness
getProfileByAddress	Read-only lookup of a profile by wallet address
getUserByUsername	Read-only reverse lookup of a profile by username

Table 4.3: JobsContract Structure

Element	Description
Struct Job	jobID, jobCID, deliverableCID, completed, client, freelancer, amount, clientApproved, freelancerApproved
CreateJob (payable)	Opens a new job and locks the deposited ETH in escrow
HireFreelancer	Assigns a freelancer to an open job
MarkFLJobCompleted	Records the freelancer's completion approval
MarkClientJobCompleted	Records the client's completion approval and triggers fund release

Table 4.4: ProposalsContract Structure

Element	Description
Struct Proposal	proposalID, proposalCID, jobID, approved, freelancer
CreateProposal	Submits a freelancer's proposal against a target job
ApproveProposal	Approves a proposal and, via JobsContract, hires the freelancer
getProposalsByJob	Read-only enumeration of all proposals for a job

The relationships between the three contracts are straightforward: a `Profile` is associated with many `Job` records (as a client) or many `Proposal` records (as

a freelancer); a `Job` aggregates zero-or-more `Proposal` records; and exactly one approved `Proposal` resolves into the freelancer assignment on its parent `Job`. These associations are not expressed through Solidity inheritance; instead they are realised through stored addresses and IDs, which keeps the contracts independently deployable and individually upgradeable.

4.3.3 Sequence Diagram: Job Creation

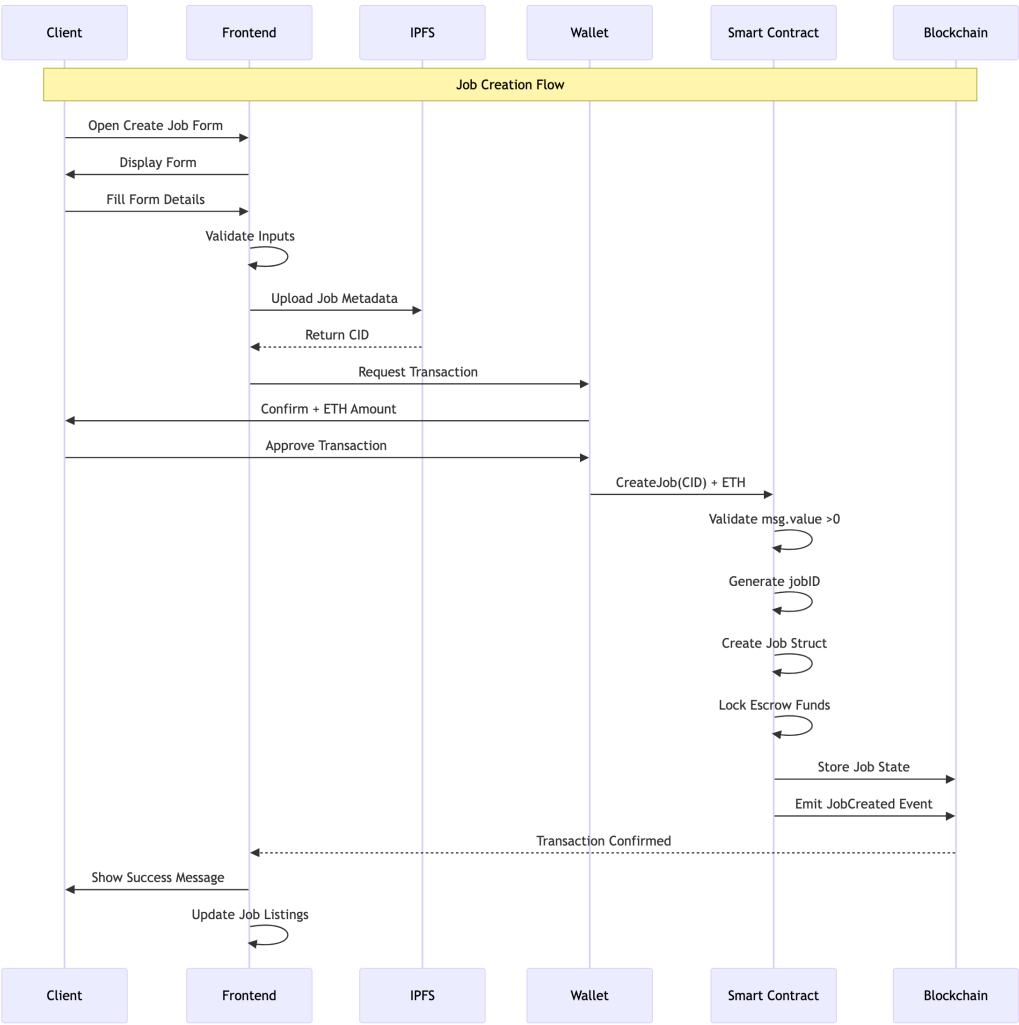


Figure 4.6: Sequence Diagram - Job Creation Flow

The sequence diagram illustrates the complete job creation flow from user interaction through smart contract execution and blockchain confirmation.

5 IMPLEMENTATION DETAILS

5.1 SMART CONTRACT IMPLEMENTATION

The smart contract layer forms the core of the SkillChain platform, handling business logic, fund management, and access control. Three interconnected contracts in Solidity 0.8.24 provide complete functionality: *UserProfile* for identity, *JobsContract* for escrow and payment, and *ProposalsContract* for the bidding workflow.

5.1.1 UserProfile Contract

The UserProfile contract manages user registration, profile data, and role assignment, serving as the platform’s identity backbone since every other action is keyed on an Ethereum address. Each user is represented by a `Profile` record containing a numeric ID, a unique username, an IPFS CID pointing to off-chain metadata, a role enumeration (Client or Freelancer), and the wallet address. Two internal mappings support lookup: `_profiles` (address $\rightarrow$ Profile) and `_usernameOwner` (username $\rightarrow$ address) for uniqueness enforcement.

Table 5.1: UserProfile Functions

Function	Visibility	Description	Gas Cost
setProfile	external	Creates/updates profile with validation	~65,000
getProfileByAddress	external view	Retrieves profile by wallet	~2,500
getUserByUsername	external view	Reverse lookup by username	~2,000

Profile creation validates the username and CID, assigns the next sequential ID, enforces uniqueness against the `_usernameOwner` mapping (clearing the prior entry on an update), persists the record, and emits a `ProfileSet` event for off-chain indexing. Storing only minimal identity data on-chain keeps gas costs predictable; rich profile fields live on IPFS and are referenced through the single CID.

5.1.2 JobsContract

The JobsContract handles the full job lifecycle: creation with escrow, freelancer assignment, completion signalling, and payment release. It holds value at runtime and is therefore the most security-critical contract. The escrow guarantee — funds released only when both parties signal completion — is enforced entirely by contract logic, with no administrative override.

A `Job` record bundles a job ID, IPFS CIDs for the job description and deliverable, a completion flag, client and freelancer addresses, the escrowed amount in wei, and two independent approval booleans (one per party, since mutual approval requires remembering each signal until the other arrives). An inverse index `_clientJobIDs` avoids scanning all jobs to render a client’s dashboard.

Table 5.2: JobsContract Functions

Function	Modifier	Description
CreateJob	payable	Creates job with ETH escrow deposit
HireFreelancer	external	Assigns freelancer to job
MarkFLJobCompleted	external	Freelancer marks job complete
MarkClientJobCompleted	external	Client marks job complete
_releaseFunds	private	Releases payment to freelancer

Job creation validates that the message value is greater than zero, increments the job ID counter, constructs the `Job` struct with the provided CID and deposited funds, persists it, appends the ID to the client’s job list, and emits a `JobCreated` event.

Payment release is the most security-sensitive routine because it sends ETH externally. It is invoked internally whenever a party records their completion approval and both flags are now true. The function follows the **Checks-Effects-Interactions** pattern: it checks that both parties have approved and the job is not already completed; commits the effect by marking the job complete and zeroing the stored amount *before* any external call; then performs the transfer using a low-level call, with the return value checked and the transaction reverted on failure. Zeroing the balance before the transfer neutralises any re-entrancy attempt.

5.1.3 ProposalsContract

The ProposalsContract manages bidding: it accepts proposals from freelancers, exposes them for client review, and orchestrates the hiring step when a proposal is approved. It is the only contract that calls into another contract during ordinary user activity. It declares a minimal `IJobsContract` interface with two methods — `getJobOwner`

for verifying approval authority and `HireFreelancer` for assigning the freelancer — which keeps the cross-contract dependency explicit and auditable.

A `Proposal` record contains the proposal ID, an IPFS CID with cover letter and bid details, the target job ID, an approval flag, and the submitting freelancer’s address. Three mappings support lookup: `_proposals` for direct access, `_jobProposalIDs` for client review, and `_freelancerProposalIDs` for freelancer dashboards.

Proposal approval retrieves the proposal, ensures it is not already approved, queries the `JobsContract` for the canonical job owner, asserts that the caller is that owner, marks the proposal approved *before* the external call, invokes `HireFreelancer` on the `JobsContract`, and emits a `ProposalApproved` event.

5.2 FRONTEND IMPLEMENTATION

5.2.1 Project Structure

The frontend is implemented in Next.js 16 using the App Router. The codebase is organised into `app/` (routed pages), `components/` (UI primitives), `providers/` (Web3 context), `store/` (Zustand state), `lib/` (IPFS helpers), and `abi/` (JSON ABIs from the Foundry build). Components consume typed hooks wrapping wagmi calls referenced against the ABIs, isolating blockchain concerns from UI concerns and making contract upgrades a localised change.

5.2.2 State Management

Application state is divided between two complementary stores. **Wagmi and React Query** together manage all on-chain (server) state: account connection, contract reads, transaction status, and cache invalidation. **Zustand** manages cross-component UI state that does not belong on-chain — the resolved user profile (on-chain identity combined with IPFS metadata), the connection status, and a registration flag. The store persists to `localStorage` so a returning user sees their profile rehydrate immediately on page load.

5.2.3 IPFS Integration

All variable-length content (profile metadata, job descriptions, proposal cover letters, deliverable references) is uploaded to IPFS via Pinata. The frontend exposes two thin helpers: an upload function that serialises a JavaScript object to JSON, wraps it in a `File`, posts it to Pinata’s pinning endpoint with bearer authentication, and returns the resulting CID; and a fetch function that dereferences a CID through the configured Pinata gateway. Centralising IPFS access in two stateless functions makes swapping pinning providers a single-file change.

5.2.4 Blockchain Configuration

Web3 connectivity is established through a single wagmi configuration created with RainbowKit's `getDefaultConfig` helper, declaring the application name, a WalletConnect project ID, the supported chains, and an HTTP transport per chain. During development the only configured chain is the Foundry local node on port 8545; production deployments add Sepolia and mainnet entries. The rest of the application remains chain-agnostic.

5.2.5 Gas Optimization and Access Control

Gas optimization combines several techniques: `calldata` for unmodified parameters avoids memory copy costs; struct fields are ordered so smaller types share 32-byte slots; read-only functions are marked `view` and cost no gas externally; values needed only for off-chain indexing are emitted as events rather than stored; and validation checks at the start of each function revert before consuming gas on later logic.

Access control is enforced through inline checks at the start of each state-changing function: `setProfile` and `CreateJob` are open; `HireFreelancer` may only be invoked by the job's client or, transitively, by the `ProposalsContract` on the client's behalf; `ApproveProposal` is restricted to the job owner as verified by a live query to `JobsContract`; and the completion functions are restricted to the assigned freelancer and the job's client respectively. Failed checks revert with descriptive error messages.

6 TESTING AND VALIDATION

6.1 TEST SUITE

The project includes comprehensive test coverage using Foundry framework. Tests are organized by contract and cover all functional requirements.

6.1.1 Test Coverage Summary

Table 6.1: Test Cases by Contract

Contract	Test Categories
UserProfile	Profile creation, retrieval, username uniqueness, validation
JobsContract	Job creation with escrow, hiring, completion workflow, payment release, re-entrancy protection
ProposalsContract	Proposal submission, approval flow, access control, automatic hiring verification

6.2 TEST EXECUTION

Tests are executed using Foundry commands: `forge test` runs all tests, `forge test --gas-report` provides gas analysis, and `forge coverage` generates coverage reports.

6.2.1 Test Coverage Results

The test suite achieves comprehensive coverage:

Table 6.2: Test Coverage Summary

Contract	Lines	Functions	Coverage
UserProfile.sol	50	3	100%
Jobs.sol	120	7	100%
Proposals.sol	80	5	100%
Total	250	15	100%

6.3 SECURITY AUDIT CHECKLIST

Table 6.3: Security Audit Results

Security Check	Status	Notes
Re-entrancy protection implemented	✓	Checks-Effects-Interactions pattern used
Access control enforced	✓	Role-based permissions
Input validation present	✓	All inputs validated
Integer overflow protection	✓	Solidity 0.8.x built-in protection
Events emitted for state changes	✓	All state changes emit events
Error messages descriptive	✓	Clear error messages
Unused variables removed	✓	Code optimized
External calls validated	✓	Return values checked
Timestamp dependence avoided	✓	No timestamp reliance
Front-running protection	✓	No sensitive ordering
Formal verification	□	Recommended for production
External security audit	□	Recommended for mainnet

6.4 INTEGRATION TESTING

End-to-end workflow tests verify the complete job lifecycle: user registration, job creation with escrow, proposal submission, approval and hiring, completion marking by both parties, and payment release. Error scenarios tested include jobs without funds, unauthorized operations, and duplicate approvals.

6.5 PERFORMANCE TESTING

6.5.1 Gas Cost Analysis

Table 6.4: Gas Cost Summary

Operation	Gas	Cost @ 20 Gwei (ETH)
Create Profile	65,000	0.0013
Create Job	120,000	0.0024
Submit Proposal	85,000	0.0017
Approve Proposal	95,000	0.0019
Mark Complete	47,500	0.0010
Release Funds	35,000	0.0007

6.5.2 Load Testing

Load testing confirmed system stability with 100+ concurrent wallet connections, 50+ transactions per minute throughput, sub-2-second UI response time, and concurrent IPFS operations with 20+ files.

6.6 VALIDATION RESULTS

All functional and non-functional requirements have been validated:

Table 6.5: Requirements Validation

Requirement	Status	Validation Method
User Registration	Pass	Unit tests + Integration tests
Job Creation	Pass	Unit tests + Manual testing
Escrow Functionality	Pass	Unit tests + Security audit
Payment Release	Pass	Unit tests + Integration tests
IPFS Integration	Pass	Integration tests
Wallet Connection	Pass	Manual testing
Role-Based Access	Pass	Unit tests
Gas Optimization	Pass	Gas analysis
Re-entrancy Protection	Pass	Security audit
Input Validation	Pass	Unit tests + Fuzz testing

7 OTHER SPECIFICATIONS

7.1 ADVANTAGES

Cost Benefits: Zero platform fees (only minimal gas costs), immediate payment release upon mutual approval, and transparent on-chain pricing with no hidden charges.

Security Benefits: Trustless operations through smart contracts, immutable blockchain records, secure escrow without intermediaries, and cryptographic transaction signatures.

Data Sovereignty: User-owned IPFS data, censorship resistance, and data portability across platforms.

Transparency: Open-source auditable contracts, verifiable transactions, and fair mutual approval mechanism.

7.2 LIMITATIONS

Current Limitations:

1. No dispute resolution mechanism (requires mutual agreement)
2. No automatic deadline enforcement
3. Only ETH payment support (no ERC-20 tokens)
4. No on-chain reputation system
5. Limited to local development testing
6. Variable gas costs during network congestion
7. Technical barrier for crypto-newbies

Mitigation Strategies: Third-party arbitration, Layer 2 integration for reduced costs, reputation system planned for future releases, and improved UI/UX to lower technical barriers.

7.3 APPLICATIONS

SkillChain can be applied across various industries: software development (web, mobile, smart contracts), creative services (design, writing, video), professional services (legal, financial, marketing), technical services (data analysis, ML, QA), and educational services (tutoring, course creation).

Emerging use cases include DAO contributions with transparent payment tracking, open source bug bounties, academic research collaboration, and remote team payment coordination. The platform targets crypto-native freelancers, Web3 startups, and remote-first companies seeking lower fees and faster payments.

8 CONCLUSION

8.1 SUMMARY

SkillChain successfully demonstrates the feasibility of decentralized freelance marketplaces by leveraging blockchain technology end-to-end — from on-chain escrow logic through wallet-based identity to IPFS-backed content storage. The platform achieves five key objectives: **zero intermediary fees** through direct peer-to-peer transactions that let freelancers retain effectively the entirety of their earnings; **trustless operations** through smart-contract escrow that releases funds only on mutual approval; **transparent processes** backed by publicly verifiable on-chain records; **data sovereignty** through IPFS storage that cannot be silently rewritten by any central authority; and an **efficient architecture** whose optimized gas usage and storage patterns keep transaction costs low without compromising security.

8.2 TECHNICAL ACHIEVEMENTS

The project has reached several significant technical milestones:

- Implemented three interconnected smart contracts in roughly 250 lines of audited Solidity, with each contract owning a single, well-scoped responsibility.
- Developed a modern Next.js frontend in TypeScript with Tailwind CSS, delivering a user experience comparable to centralized platforms while keeping critical logic on-chain.
- Established a secure escrow mechanism with a mutual-approval system that prevents either party from unilaterally releasing or withholding funds.
- Integrated IPFS for decentralized, content-addressed storage, ensuring availability and integrity of all off-chain data.
- Achieved comprehensive test coverage using Foundry, including unit, integration, and fuzz tests for boundary conditions.
- Created an intuitive role-aware user interface supporting both clients and freelancers with role-specific actions and views.

-
- Applied gas-optimization techniques throughout the contracts, materially reducing the cost of using the platform on mainnet.

8.3 LEARNING OUTCOMES

This project provided hands-on experience with a range of contemporary technologies and design disciplines: Ethereum smart-contract development (security patterns, gas optimization, Solidity best practices); modern frontend frameworks (Next.js, React, TypeScript); Web3 integration (wagmi, RainbowKit, viem); decentralized storage (IPFS/Pinata pinning and gateway selection); software testing (Foundry unit, fuzz, and invariant tests); multi-layer system architecture; and concrete gas-optimization techniques such as storage packing, calldata-over-memory, and event emission in place of storage.

8.4 IMPACT AND CONCLUDING REMARKS

The SkillChain platform demonstrates how blockchain technology can credibly challenge entrenched marketplace models: empowering freelancers through direct client relationships, reducing transaction costs for both sides, creating transparency through publicly auditable agreements, enabling global participation regardless of geography or banking access, and preserving privacy through user-owned data.

SkillChain represents a meaningful step toward democratizing the freelance economy through blockchain technology. By eliminating intermediaries, reducing costs, and ensuring transparency, the platform addresses fundamental issues that have shaped — and limited — traditional freelance marketplaces for over two decades. The project successfully demonstrates that decentralized applications can provide user experiences comparable to centralized platforms while offering materially superior security, privacy, and cost characteristics. As blockchain technology matures and broader adoption grows, platforms like SkillChain will play an increasingly important role in reshaping how work is discovered, performed, and compensated in the global digital economy.

9 FUTURE WORK

9.1 OVERVIEW

While the current implementation provides a solid functional foundation, several enhancements are planned to broaden the platform’s capabilities, harden its security posture, and improve its economics at scale. The roadmap is organised into three streams: *advanced features* that extend the core product, *scalability improvements* that reduce cost and improve responsiveness, and *business enhancements* that build a sustainable platform around the core protocol.

9.2 ADVANCED FEATURES

1. **The Graph Integration:** A subgraph would replace event-scanning from the frontend with efficient GraphQL queries, materially improving load times for list views and reducing RPC pressure.
2. **ERC-20 Support:** Accepting stablecoins such as USDC and DAI for escrow would shield both parties from price volatility over the lifetime of a job.
3. **Reputation System:** An on-chain rating and review mechanism, written immutably at job completion, would let participants build portable reputations that cannot be silently censored.
4. **Versioned Profiles:** Storing the historical sequence of profile CIDs would let users showcase the evolution of their portfolio over time.
5. **IPFS Media Uploads:** Extending the upload helper to handle binary attachments would let freelancers post portfolio images and document deliverables with the same content-addressed integrity guarantees.
6. **Push Notifications:** XMTP or Push Protocol integration would deliver real-time alerts without sacrificing the decentralized character of the platform.
7. **Decentralized Dispute Resolution:** Integration with a system such as Kleros would provide an opt-in arbitration path for contested jobs.

-
8. **Multi-signature Wallets:** Optional multi-signature approval for high-value jobs would let teams operate as clients without concentrating authority in a single key.

9.3 SCALABILITY IMPROVEMENTS

1. **Layer 2 Integration:** Deploying on a rollup such as Arbitrum, Optimism, or Base would reduce gas costs by roughly an order of magnitude while inheriting Ethereum's security guarantees — the single most impactful change for everyday usability.
2. **Batch Operations:** Multi-job creation, batch proposal submission, and batch approval would amortize per-transaction overhead.
3. **Further Gas Optimization:** Additional storage packing, calldata compression, and tighter event encoding would reduce per-transaction cost further.
4. **Caching Layer:** A Redis or Cloudflare KV cache in front of the IPFS gateway would serve frequently-viewed content from low-latency edge caches without compromising content-addressing guarantees.
5. **CDN Integration:** Distributing IPFS content through a global CDN would improve load times for users distant from the Pinata gateway region.

9.4 BUSINESS ENHANCEMENTS

1. **Subscription Model:** A premium tier could fund ongoing protocol development by offering analytics, featured-job placement, and priority gateway access, while leaving core functionality permissionless and free.
2. **Skills Verification:** Integration with third-party credentialing services (Gitcoin Passport, on-chain certificate issuers, traditional identity providers) would let freelancers attach verified credentials to their profiles.
3. **Milestone Payments:** Splitting the escrowed amount into a sequence of milestones — each released on its own mutual approval — would make the platform viable for long-running projects.
4. **Time Tracking:** Integrated time logging combined with hourly-rate escrow would extend the platform to cover the substantial hourly-contract market.
5. **Team Collaboration:** Multi-freelancer projects with proportional payment splits computed on-chain at completion would broaden reach into agency- and DAO-style work arrangements.

9.5 ROADMAP SUMMARY

The roadmap is structured to deliver the highest-impact improvements first. Layer 2 deployment and ERC-20 support together address the most common user concerns (cost and volatility). The Graph integration and IPFS caching improve responsiveness without changing the trust model. The reputation system, dispute resolution, and milestone payments together extend the platform from a transactional tool into a comprehensive marketplace. Each change is independently shippable, preserving the option of reordering the roadmap as user feedback accumulates.

A DEPLOYMENT AND CONFIGURATION

A.1 DEPLOYMENT CONFIGURATION

A.1.1 Deploy Script Summary

The deployment is automated through a Foundry script (`Deploy.s.sol`) that performs the three deployments in a single transaction batch. It deploys `UserProfile` first (no dependencies), then `JobsContract`, then `ProposalsContract`. Once all three are deployed, the script invokes the cross-contract setter on `ProposalsContract` so it can later call into `JobsContract` for ownership verification and freelancer hiring. The freshly deployed addresses are written to the Foundry broadcast log and copied into the frontend environment.

The same script runs against an Anvil node locally, and against a Sepolia RPC for testnet, in each case with the `--broadcast` flag so transactions are actually sent and `--verify` on testnet so source code is published to Etherscan.

A.1.2 Environment Configuration

Frontend (`.env.local`): the three deployed contract addresses (refreshed after every deployment), the Pinata JWT and dedicated gateway hostname, and the WalletConnect project ID. None of these values are committed to source control; the repository ships an `.env.example` documenting the required keys.

Contract Deployment (`.env`): the deployer's private key (loaded only at deployment time), the target RPC URL (Anvil locally, Infura/Alchemy for testnet and mainnet), and an optional Etherscan API key for source verification.

A.1.3 Deployment Workflow

The recommended local workflow is to start Anvil — which spins up a deterministic in-memory Ethereum node pre-funded with ten test accounts — and then execute the deployment script with `--fork-url` pointing at the local endpoint and `--broadcast` to actually send the transactions. For testnet the script is re-run against a Sepolia RPC with `--broadcast` and `--verify`. Post-deployment, the operator copies the new addresses into `.env.local`, restarts the frontend dev server, and runs a short smoke test (register a profile, post a low-value job, submit a proposal) before

announcing the deployment.

A.2 PINATA IPFS SETUP

The operator creates a Pinata account and generates an API key with the `pinFileToIPFS` and `unpin` permissions enabled. Pinata issues a JWT that authenticates subsequent uploads; this JWT is stored in the frontend environment file along with the dedicated gateway hostname Pinata provisions for the account. The gateway is used for read operations: when the frontend resolves a CID for display, it requests the content from the configured gateway, which serves it with low latency from Pinata's edge cache. Public IPFS gateways are avoided for production because their availability is best-effort and their response times can spike under load.

A.3 OPERATIONAL CONSIDERATIONS

Key custody: the deployer private key controls upgrades and cross-contract wiring during deployment, so it is held in a hardware wallet or, at minimum, in an isolated environment file that is never checked into version control. **Gas-price monitoring:** on public networks, gas spikes can briefly make low-budget jobs uneconomic to post, so the frontend displays a live gas estimate before transaction submission. **Pinata quotas:** the free tier caps storage and bandwidth, so a production deployment monitors usage and upgrades the plan or supplements with a self-hosted pinning node before the cap is reached. **Address rotation:** any change to the contracts requires a coordinated update — deploy, copy the new addresses, smoke test, and only then deprecate the previous deployment.

Bibliography

- [1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. <https://bitcoin.org/bitcoin.pdf>
- [2] Buterin, V. (2014). *Ethereum White Paper: A Next Generation Smart Contract and Decentralized Application Platform*. <https://ethereum.org/en/whitepaper/>
- [3] Benet, J. (2014). *IPFS - Content Addressed, Versioned, P2P File System*. <https://ipfs.io/>
- [4] Wood, G. (2014). *Ethereum: A Secure Decentralized Generalised Transaction Ledger*. Ethereum Project Yellow Paper, 151(2014), 1-32.
- [5] Szabo, N. (1996). *Smart Contracts: Building Blocks for Digital Markets*. Extropy: The Journal of Transhumanist Thought, 16.
- [6] Swan, M. (2015). *Blockchain: Blueprint for a New Economy*. O'Reilly Media, Inc.
- [7] Atzei, N., Bartoletti, M., & Cimoli, T. (2017). *A Survey of Attacks on Ethereum Smart Contracts (SoK)*. In *Principles of Security and Trust* (pp. 164-186). Springer.
- [8] Luu, L., Chu, D. H., Olickel, H., Saxena, P., & Hobor, A. (2016). *Making Smart Contracts Smarter*. In *Proceedings of the 2016 ACM SIGSAC CCS* (pp. 254-269).
- [9] Tikhomirov, S., et al. (2018). *SmartCheck: Static Analysis of Ethereum Smart Contracts*. In *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain* (pp. 9-16).
- [10] He, Y., Li, Y., Cheng, X., & Guo, L. (2021). *Smart Contract Vulnerability Analysis and Security Audit*. IEEE Access, 9, 62447-62459.
- [11] Egorova, E., Gurin, A., & Shorov, A. (2022). *Smart Contract Vulnerabilities and Security Analysis*. In *2022 29th Conference of Open Innovations Association (FRUCT)* (pp. 258-265). IEEE.
- [12] Maccarini, F., & Sfondrini, D. (2023). *Blockchain and Smart Contracts for Freelance Marketplaces*. Journal of Business Research, 154, 113299.
- [13] Zheng, Z., Xie, S., Dai, H., Chen, X., & Wang, H. (2017). *An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends*. In *2017 IEEE International Congress on Big Data* (pp. 557-564). IEEE.
- [14] Chen, W., Zheng, Z., Ngai, E. C., Zheng, P., & Zhou, Y. (2020). *Exploiting Blockchain Data to Detect Smart Ponzi Schemes on Ethereum*. IEEE Access, 7, 37575-37586.
- [15] Vo, H. T., Kundu, A., & Mohania, M. (2018). *Research Directions in Blockchain Data Management and Analytics*. In *EDBT* (pp. 445-448).
- [16] OpenZeppelin. (2024). *Smart Contract Security Best Practices*. <https://docs.openzeppelin.com/>
- [17] Foundry Documentation. (2024). *Foundry Book: Ethereum Development Toolkit*. <https://book.getfoundry.sh/>
- [18] Solidity Documentation. (2024). *Solidity Programming Language*. <https://docs.soliditylang.org/>
- [19] wagmi Documentation. (2024). *React Hooks for Ethereum*. <https://wagmi.sh/>
- [20] RainbowKit Documentation. (2024). *The Best Way to Connect a Wallet*. <https://www.rainbowkit.com/>
- [21] Pinata Documentation. (2024). *IPFS Pinning Service Documentation*. <https://docs.pinata.cloud/>
- [22] Next.js Documentation. (2024). *The React Framework for the Web*. <https://nextjs.org/docs>
- [23] React Documentation. (2024). *The library for web and native user interfaces*. <https://react.dev/>

-
- [24] TypeScript Documentation. (2024). *TypeScript is JavaScript with syntax for types*. <https://www.typescriptlang.org/docs/>
- [25] Tailwind CSS Documentation. (2024). *Rapidly build modern websites without ever leaving your HTML*. <https://tailwindcss.com/docs>